Trip Auditor Documentation

Generated by Doxygen 1.14.0

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Chapter 4

Namespace Documentation

4.1 app Namespace Reference

Classes

- class AIReportGenerator
- class TripAuditorApp
- class UserManager

Variables

- app = TripAuditorApp()

4.1.1 Detailed Description

```
--------------------------------------------------------------------------------------------------------------
Start of Imports and Setup
--------------------------------------------------------------------------------------------------------------
```

4.1.2 Variable Documentation

4.1.2.1 app

```
app.app = TripAuditorApp()
```

Definition at line 340 of file app.py.

Chapter 5

Class Documentation

5.1 app.AIReportGenerator Class Reference

Static Public Member Functions

- generate (filtered_df)

5.1.1 Detailed Description

Definition at line 78 of file app.py.

5.1.2 Member Function Documentation

5.1.2.1 generate()

```
app.AIReportGenerator.generate (
              filtered_df)  [static]
```

Generate a summary AI report from the filtered DataFrame.

Definition at line 81 of file app.py.

```
00081     def generate(filtered_df):
00082         """Generate a summary AI report from the filtered DataFrame."""
00083         if filtered_df.empty:
00084             return "No data available for AI report."
00085         most_profitable_vehicle = filtered_df.groupby('Vehicle ID')['Net Profit'].sum().idxmax()
00086         top_routes = ", ".join(filtered_df['Route'].value_counts().head(2).index) if 'Route' in
      filtered_df.columns else "N/A"
00087         avg_profit_per_trip = round(filtered_df['Net Profit'].sum() / len(filtered_df), 2)
00088         rev = filtered_df['Freight Amount'].sum()
00089         exp = filtered_df['Total Trip Expense'].sum()
00090         profit = filtered_df['Net Profit'].sum()
00091         kms = filtered_df['Actual Distance (KM)'].sum()
00092         profit_pct = round((profit / rev * 100), 1) if rev else 0
00093         per_km = round(profit / kms, 2) if kms else 0
00094         return f"""
00095 AI Report Highlights:
00096
00097 Total Trips: {len(filtered_df)}
00098 On-going Trips: {filtered_df[filtered_df['Trip Status'] == 'Pending Closure'].shape[0]}
00099 Completed Trips: {filtered_df[filtered_df['Trip Status'] == 'Completed'].shape[0]}
```

```
00100 Profit Percentage: {profit_pct}%
00101
00102 Financials:
00103 - Revenue: Rs{round(rev / 1e6, 2)}M
00104 - Expense: Rs{round(exp / 1e6, 2)}M
00105 - Profit: Rs{round(profit / 1e6, 2)}M
00106 - KMs Travelled: {round(kms / 1e3, 1)}K
00107 - Cost per KM: Rs{per_km}
00108
00109 AI Insights:
00110 - Top Vehicle: {most_profitable_vehicle}
00111 - Average Profit per Trip: Rs{avg_profit_per_trip}
00112 - Top Routes: {top_routes}
00113 """
00114
00115 """
00116
      ------------------------------------------------------------------------------------------------------------------------

00117 Main Application Class
00118
      ------------------------------------------------------------------------------------------------------------------------

00119 [Summary] TripAuditorApp is the main Flask application for managing trip audits.
00120 [Details] It initializes the Flask app, loads datasets, manages user sessions, and defines all routes
      for the application.
00121
00122 """
00123
```

The documentation for this class was generated from the following file:

- D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py

5.2 app.TripAuditorApp Class Reference

Public Member Functions

- __init__ (self)
- load_fleet_data (self)
- load_closure_data (self)
- register_routes (self)
- run (self)

Public Attributes

- app = Flask(__name__)
- str fleet_file = 'fleet_50_entries.xlsx'
- str closure_file = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
- df = self.load_fleet_data()
- closure_df = self.load_closure_data()
- vehicles = sorted(self.df['Vehicle ID'].dropna().unique())
- str routes = sorted(self.df['Route'].dropna().unique()) if 'Route' in self.df.columns else [ ]
- user_manager = UserManager(os.path.join(os.getcwd(), 'users.json'))

5.2.1 Detailed Description

Definition at line 124 of file app.py.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 __init__()

```
app.TripAuditorApp.__init__ (
            self)
```

Definition at line 126 of file app.py.

```
00126     def __init__(self):
00127         # Initialize Flask app
00128         self.app = Flask(__name__)
00129         self.app.secret_key = 'supersecret'
00130 
00131         # Load datasets
00132         self.fleet_file = 'fleet_50_entries.xlsx'
00133         self.closure_file = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
00134         self.df = self.load_fleet_data()
00135         self.closure_df = self.load_closure_data()
00136 
00137         # Prepare vehicle and route lists
00138         self.vehicles = sorted(self.df['Vehicle ID'].dropna().unique())
00139         self.routes = sorted(self.df['Route'].dropna().unique()) if 'Route' in self.df.columns else []
00140 
00141         # User management
00142         self.user_manager = UserManager(os.path.join(os.getcwd(), 'users.json'))
00143 
00144         # Register all routes
00145         self.register_routes()
00146 
```

5.2.3 Member Function Documentation

5.2.3.1 load_closure_data()

```
app.TripAuditorApp.load_closure_data (
            self)
```

Load and preprocess the closure Excel data.

Definition at line 157 of file app.py.

```
00157     def load_closure_data(self):
00158         """Load and preprocess the closure Excel data."""
00159         df = pd.read_excel(self.closure_file)
00160         df.columns = df.columns.str.strip()
00161         df['Trip Date'] = pd.to_datetime(df['Trip Date'], errors='coerce')
00162         df['Day'] = df['Trip Date'].dt.day
00163         return df
00164 
```

5.2.3.2 load_fleet_data()

```
app.TripAuditorApp.load_fleet_data (
            self)
```

Load and preprocess the fleet Excel data.

Definition at line 148 of file app.py.

```
00148     def load_fleet_data(self):
00149         """Load and preprocess the fleet Excel data."""
00150         df = pd.read_excel(self.fleet_file)
00151         df.columns = df.columns.str.strip()
00152         df['Trip Date'] = pd.to_datetime(df['Trip Date'], errors='coerce')
00153         df['Day'] = df['Trip Date'].dt.day
00154         return df
00155 
```

5.2.3.3 register_routes()

```
app.TripAuditorApp.register_routes (
              self)
```

Define all Flask routes for the app.

Definition at line 166 of file app.py.

```
00166     def register_routes(self):
00167         """Define all Flask routes for the app."""
00168 
00169         @self.app.route('/')
00170         def home():
00171             # Redirect to signup page
00172             return redirect(url_for('signup'))
00173 
00174         @self.app.route('/signup', methods=['GET', 'POST'])
00175         def signup():
00176             # Handle user signup
00177             if request.method == 'POST':
00178                 email = request.form['email']
00179                 if self.user_manager.email_exists(email):
00180                     return render_template('signup.html', error="Email already registered!")
00181                 self.user_manager.add_user(
00182                     name=request.form['fullname'],
00183                     email=email,
00184                     password=request.form['password']
00185                 )
00186                 return redirect(url_for('login'))
00187             return render_template('signup.html')
00188 
00189         @self.app.route('/login', methods=['GET', 'POST'])
00190         def login():
00191             # Handle user login
00192             if request.method == 'POST':
00193                 user = self.user_manager.find_user(request.form['email'])
00194                 if user and check_password_hash(user['password'], request.form['password']):
00195                     session['user'] = user
00196                     return redirect(url_for('dashboard'))
00197                 return render_template('login.html', error="Invalid credentials.")
00198             return render_template('login.html')
00199 
00200         @self.app.route('/dashboard')
00201         def dashboard():
00202             # Main dashboard with filters and summary
00203             if 'user' not in session:
00204                 return redirect(url_for('login'))
00205             vehicle = request.args.get('vehicle')
00206             route = request.args.get('route')
00207             filtered = self.df.copy()
00208             if vehicle:
00209                 filtered = filtered[filtered['Vehicle ID'] == vehicle]
00210             if route:
00211                 filtered = filtered[filtered['Route'] == route]
00212 
00213             total_trips = len(filtered)
00214             ongoing = filtered[filtered['Trip Status'] == 'Pending Closure'].shape[0]
00215             closed = filtered[filtered['Trip Status'] == 'Completed'].shape[0]
00216             flags = filtered[filtered['Trip Status'] == 'Under Audit'].shape[0]
00217             resolved = filtered[(filtered['Trip Status'] == 'Under Audit') & (filtered['POD Status']
      == 'Yes')].shape[0]
00218 
00219             rev = filtered['Freight Amount'].sum()
00220             exp = filtered['Total Trip Expense'].sum()
00221             profit = filtered['Net Profit'].sum()
00222             kms = filtered['Actual Distance (KM)'].sum()
00223 
00224             rev_m = round(rev / 1e6, 2)
00225             exp_m = round(exp / 1e6, 2)
00226             profit_m = round(profit / 1e6, 2)
00227             kms_k = round(kms / 1e3, 1)
00228             per_km = round(profit / kms, 2) if kms else 0
00229             profit_pct = round((profit / rev) * 100, 1) if rev else 0
00230 
00231             daily = filtered.groupby('Day')['Trip ID'].count().reindex(range(1, 32),
      fill_value=0).tolist()
00232             audited = filtered[filtered['Trip Status'] == 'Under Audit'].groupby('Day')['Trip
      ID'].count().reindex(range(1, 32), fill_value=0).tolist()
00233             audit_pct = [round(a / b * 100, 1) if b else 0 for a, b in zip(audited, daily)]
00234 
```

```
00235             bar_labels = ['Revenue', 'Expense', 'Profit']
00236             bar_values = [float(rev_m), float(exp_m), float(profit_m)]
00237             ai_report = AIReportGenerator.generate(filtered)
00238 
00239             return render_template('dashboard.html',
00240                 total_trips=total_trips, ongoing=ongoing, closed=closed,
00241                 flags=flags, resolved=resolved, rev_m=rev_m, exp_m=exp_m,
00242                 profit_m=profit_m, kms_k=kms_k, per_km=per_km, profit_pct=profit_pct,
00243                 ai_report=ai_report, vehicles=self.vehicles, routes=self.routes,
00244                 selected_vehicle=vehicle, selected_route=route,
00245                 daily=daily, audited=audited, audit_pct=audit_pct,
00246                 bar_labels=bar_labels, bar_values=bar_values)
00247 
00248         @self.app.route('/trip-generator')
00249         def trip_generator():
00250             # Show all trips
00251             data = self.df[['Trip ID', 'Vehicle ID', 'Trip Status']]
00252             return render_template('table_page.html', title="Trip Generator",
    table=data.to_html(classes='text-white', index=False))
00253 
00254         @self.app.route('/trip-closure')
00255         def trip_closure():
00256             # Show trips pending closure
00257             data = self.df[self.df['Trip Status'] == 'Pending Closure'][['Trip ID', 'Vehicle ID',
    'Trip Status']]
00258             return render_template('table_page.html', title="Trip Closure",
    table=data.to_html(classes='text-white', index=False))
00259 
00260         @self.app.route('/trip-auditor')
00261         def trip_auditor():
00262             # Show trips under audit
00263             data = self.df[self.df['Trip Status'] == 'Under Audit'][['Trip ID', 'Vehicle ID', 'POD
    Status']]
00264             return render_template('table_page.html', title="Trip Auditor",
    table=data.to_html(classes='text-white', index=False))
00265 
00266         @self.app.route('/trip-ongoing')
00267         def trip_ongoing():
00268             # Show ongoing trips
00269             data = self.df[self.df['Trip Status'] == 'Pending Closure'][['Trip ID', 'Vehicle ID',
    'Trip Status']]
00270             return render_template('table_page.html', title="Ongoing Trips",
    table=data.to_html(classes='text-white', index=False))
00271 
00272         @self.app.route('/trip-stats')
00273         def trip_stats():
00274             # Show trip statistics by day
00275             days = list(range(1, 32))
00276             total = self.df.groupby('Day')['Trip ID'].count().reindex(days, fill_value=0).tolist()
00277             ongoing = self.df[self.df['Trip Status'] == 'Pending Closure'].groupby('Day')['Trip
    ID'].count().reindex(days, fill_value=0).tolist()
00278             closed = self.df[self.df['Trip Status'] == 'Completed'].groupby('Day')['Trip
    ID'].count().reindex(days, fill_value=0).tolist()
00279 
00280             total_json = json.dumps(total)
00281             ongoing_json = json.dumps(ongoing)
00282             closed_json = json.dumps(closed)
00283 
00284             total_sum = sum(total)
00285             ongoing_sum = sum(ongoing)
00286             closed_sum = sum(closed)
00287 
00288             return render_template('trip_stats.html',
00289                 total_data=total_json, ongoing_data=ongoing_json, closed_data=closed_json,
00290                 total_sum=total_sum, ongoing_sum=ongoing_sum, closed_sum=closed_sum)
00291 
00292         @self.app.route('/financial-dashboard')
00293         def financial_dashboard():
00294             # Show financial dashboard with recent 10 days data
00295             df_fin = self.closure_df.copy()
00296             recent_days = sorted(df_fin['Day'].dropna().unique())[-10:]
00297             day_labels = [f"Day {int(d)}" for d in recent_days]
00298             daily = df_fin[df_fin['Day'].isin(recent_days)]
00299             revenue_data = daily.groupby('Day')['Freight Amount'].sum().reindex(recent_days,
    fill_value=0).astype(int).tolist()
00300             expense_data = daily.groupby('Day')['Total Trip Expense'].sum().reindex(recent_days,
    fill_value=0).astype(int).tolist()
00301             profit_data = [r - e for r, e in zip(revenue_data, expense_data)]
00302             total_revenue = round(df_fin['Freight Amount'].sum() / 1e6, 2)
00303             total_profit = round(df_fin['Net Profit'].sum() / 1e6, 2)
00304             total_km = round(df_fin['Actual Distance (KM)'].sum() / 1e3, 1)
00305             per_km = round(df_fin['Net Profit'].sum() / df_fin['Actual Distance (KM)'].sum(), 2) if
    df_fin['Actual Distance (KM)'].sum() else 0
00306 
00307             return render_template('financial_dashboard.html',
00308                 days=day_labels, revenue=revenue_data, expense=expense_data, profit=profit_data,
00309                 total_revenue=total_revenue, total_profit=total_profit, total_km=total_km,
```

```
      per_km=per_km)
00310
00311         @self.app.route('/logout')
00312         def logout():
00313             # Log out the current user
00314             session.pop('user', None)
00315             return redirect(url_for('login'))
00316
00317         @self.app.route('/download-summary')
00318         def download_summary():
00319             # Download AI report summary as a text file
00320             filtered = self.df
00321             report = AIReportGenerator.generate(filtered)
00322             with open("AI_Report_Summary.txt", 'w', encoding='utf-8') as f:
00323                 f.write(report)
00324             return send_file("AI_Report_Summary.txt", as_attachment=True)
00325
```

5.2.3.4 run()

```
app.TripAuditorApp.run (
              self)
```

Run the Flask app.

Definition at line 326 of file app.py.

```
00326     def run(self):
00327         """Run the Flask app."""
00328         self.app.run(host='0.0.0.0', port=7860, debug=True)
00329
00330 """
00331
      ---------------------------------------------------------------------------------------------------------------------------------------
00332 Entry Point
00333
      ---------------------------------------------------------------------------------------------------------------------------------------
00334 This is the entry point for the application, where the TripAuditorApp is created and run.
00335
00336 """
00337
```

5.2.4 Member Data Documentation

5.2.4.1 app

```
app.TripAuditorApp.app = Flask(__name__)
```

Definition at line 128 of file app.py.

5.2.4.2 closure_df

```
app.TripAuditorApp.closure_df = self.load_closure_data()
```

Definition at line 135 of file app.py.

5.2.4.3 closure_file

```
str app.TripAuditorApp.closure_file = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
```

Definition at line 133 of file app.py.

5.2.4.4 df

```
app.TripAuditorApp.df = self.load_fleet_data()
```

Definition at line 134 of file app.py.

5.2.4.5 fleet_file

```
str app.TripAuditorApp.fleet_file = 'fleet_50_entries.xlsx'
```

Definition at line 132 of file app.py.

5.2.4.6 routes

```
str app.TripAuditorApp.routes = sorted(self.df['Route'].dropna().unique()) if 'Route' in self.↩
df.columns else []
```

Definition at line 139 of file app.py.

5.2.4.7 user_manager

```
app.TripAuditorApp.user_manager = UserManager(os.path.join(os.getcwd(), 'users.json'))
```

Definition at line 142 of file app.py.

5.2.4.8 vehicles

```
app.TripAuditorApp.vehicles = sorted(self.df['Vehicle ID'].dropna().unique())
```

Definition at line 138 of file app.py.

The documentation for this class was generated from the following file:

- D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py

5.3 app.UserManager Class Reference

Public Member Functions

- __init__ (self, user_file)
- load_users (self)
- save_users (self)
- add_user (self, name, email, password, role='Owner')
- find_user (self, email)
- email_exists (self, email)

Public Attributes

- user_file = user_file
- users = self.load_users()

5.3.1 Detailed Description

Definition at line 29 of file app.py.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 __init__()

```
app.UserManager.__init__ (
               self,
               user_file)
```

Definition at line 30 of file app.py.

```
00030     def __init__(self, user_file):
00031         self.user_file = user_file
00032         self.users = self.load_users()
00033
```

5.3.3 Member Function Documentation

5.3.3.1 add_user()

```
app.UserManager.add_user (
               self,
               name,
               email,
               password,
               role = 'Owner')
```

Add a new user and save.

Definition at line 49 of file app.py.

```
00049     def add_user(self, name, email, password, role='Owner'):
00050         """Add a new user and save."""
00051         self.users.append({
00052             'name': name,
00053             'email': email,
00054             'password': generate_password_hash(password),
00055             'role': role
00056         })
00057         self.save_users()
00058
```

5.3.3.2 email_exists()

```
app.UserManager.email_exists (
               self,
               email)
```

Check if an email is already registered.

Definition at line 65 of file app.py.

```
00065     def email_exists(self, email):
00066         """Check if an email is already registered."""
00067         return any(u['email'] == email for u in self.users)
00068 
00069 """
00070 
      ------------------------------------------------------------------------------------------------------------------------
00071 AI Report Generator Class
00072 
      ------------------------------------------------------------------------------------------------------------------------
00073 [Summary] AIReportGenerator generates a summary AI report from the filtered DataFrame.
00074 [Details] Defines a static method to create a report based on the filtered DataFrame, summarizing key
      metrics like total trips, profit percentage, and financials.
00075 
00076 """
00077 
```

5.3.3.3 find_user()

```
app.UserManager.find_user (
               self,
               email)
```

Find a user by email.

Definition at line 60 of file app.py.

```
00060     def find_user(self, email):
00061         """Find a user by email."""
00062         return next((u for u in self.users if u['email'] == email), None)
00063 
```

5.3.3.4 load_users()

```
app.UserManager.load_users (
               self)
```

Load users from the JSON file, or return empty list if file doesn't exist.

Definition at line 35 of file app.py.

```
00035     def load_users(self):
00036         """Load users from the JSON file, or return empty list if file doesn't exist."""
00037         if os.path.exists(self.user_file):
00038             with open(self.user_file, 'r') as f:
00039                 return json.load(f)
00040         return []
00041 
```

5.3.3.5 save_users()

```
app.UserManager.save_users (
              self)
```

Save the current users list to the JSON file.

Definition at line 43 of file app.py.

```
00043     def save_users(self):
00044         """Save the current users list to the JSON file."""
00045         with open(self.user_file, 'w') as f:
00046             json.dump(self.users, f)
00047
```

5.3.4 Member Data Documentation

5.3.4.1 user_file

```
app.UserManager.user_file = user_file
```

Definition at line 31 of file app.py.

5.3.4.2 users

```
app.UserManager.users = self.load_users()
```

Definition at line 32 of file app.py.

The documentation for this class was generated from the following file:

- D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py

Chapter 6

File Documentation

6.1 D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py File Reference

Classes

- class app.UserManager
- class app.AIReportGenerator
- class app.TripAuditorApp

Namespaces

- namespace app

Variables

- app.app = TripAuditorApp()

6.2 D:/WorkSpace/Repositories/TripAuditor/ai_agent_1/app.py

Go to the documentation of this file.

```
00001 """
00002 
      ------------------------------------------------------------------------------------------------------------------------
00003 Start of Imports and Setup
00004 
      ------------------------------------------------------------------------------------------------------------------------
00005 """
00006 
00007 from flask import Flask, render_template, request, redirect, url_for, session, send_file
00008 from werkzeug.security import generate_password_hash, check_password_hash
00009 import pandas as pd
00010 import json
00011 import os
00012 
00013 """
00014 
      ------------------------------------------------------------------------------------------------------------------------
00015 End of Imports and Setup
00016 
      ------------------------------------------------------------------------------------------------------------------------
```

```
00017 """
00018
00019
00020 """
00021
    ------------------------------------------------------------------------------------------------------------------------
00022  User Management Class
00023
    ------------------------------------------------------------------------------------------------------------------------
00024 [Summary] UserManager handles user registration, login, and management
00025 [Details] It loads users from a JSON file, allows adding new users, and checks for existing emails.
00026
00027 """
00028
00029 class UserManager:
00030     def __init__(self, user_file):
00031         self.user_file = user_file
00032         self.users = self.load_users()
00033
00034     # This method Load users from the JSON file or return an empty list if the file doesn't exist.
00035     def load_users(self):
00036         """Load users from the JSON file, or return empty list if file doesn't exist."""
00037         if os.path.exists(self.user_file):
00038             with open(self.user_file, 'r') as f:
00039                 return json.load(f)
00040         return []
00041
00042     # This method writes the users list to the specified JSON file.
00043     def save_users(self):
00044         """Save the current users list to the JSON file."""
00045         with open(self.user_file, 'w') as f:
00046             json.dump(self.users, f)
00047
00048     # This method adds a new user to the users list and saves it to the JSON file.
00049     def add_user(self, name, email, password, role='Owner'):
00050         """Add a new user and save."""
00051         self.users.append({
00052             'name': name,
00053             'email': email,
00054             'password': generate_password_hash(password),
00055             'role': role
00056         })
00057         self.save_users()
00058
00059     # This method finds a user by email in the users list, It returns the user dictionary if found, or
    None if not found.
00060     def find_user(self, email):
00061         """Find a user by email."""
00062         return next((u for u in self.users if u['email'] == email), None)
00063
00064     # This method checks if an email already exists in the users list, It returns True if the email is
    found, or False otherwise.
00065     def email_exists(self, email):
00066         """Check if an email is already registered."""
00067         return any(u['email'] == email for u in self.users)
00068
00069 """
00070
    ------------------------------------------------------------------------------------------------------------------------
00071 AI Report Generator Class
00072
    ------------------------------------------------------------------------------------------------------------------------
00073 [Summary] AIReportGenerator generates a summary AI report from the filtered DataFrame.
00074 [Details] Defines a static method to create a report based on the filtered DataFrame, summarizing key
    metrics like total trips, profit percentage, and financials.
00075
00076 """
00077
00078 class AIReportGenerator:
00079     # This method generates a summary AI report from the filtered DataFrame.
00080     @staticmethod
00081     def generate(filtered_df):
00082         """Generate a summary AI report from the filtered DataFrame."""
00083         if filtered_df.empty:
00084             return "No data available for AI report."
00085         most_profitable_vehicle = filtered_df.groupby('Vehicle ID')['Net Profit'].sum().idxmax()
00086         top_routes = ", ".join(filtered_df['Route'].value_counts().head(2).index) if 'Route' in
    filtered_df.columns else "N/A"
00087         avg_profit_per_trip = round(filtered_df['Net Profit'].sum() / len(filtered_df), 2)
00088         rev = filtered_df['Freight Amount'].sum()
00089         exp = filtered_df['Total Trip Expense'].sum()
00090         profit = filtered_df['Net Profit'].sum()
00091         kms = filtered_df['Actual Distance (KM)'].sum()
00092         profit_pct = round((profit / rev * 100), 1) if rev else 0
00093         per_km = round(profit / kms, 2) if kms else 0
00094         return f"""
00095 AI Report Highlights:
```

```
00096
00097 Total Trips: {len(filtered_df)}
00098 On-going Trips: {filtered_df[filtered_df['Trip Status'] == 'Pending Closure'].shape[0]}
00099 Completed Trips: {filtered_df[filtered_df['Trip Status'] == 'Completed'].shape[0]}
00100 Profit Percentage: {profit_pct}%
00101
00102 Financials:
00103 - Revenue: Rs{round(rev / 1e6, 2)}M
00104 - Expense: Rs{round(exp / 1e6, 2)}M
00105 - Profit: Rs{round(profit / 1e6, 2)}M
00106 - KMs Travelled: {round(kms / 1e3, 1)}K
00107 - Cost per KM: Rs{per_km}
00108
00109 AI Insights:
00110 - Top Vehicle: {most_profitable_vehicle}
00111 - Average Profit per Trip: Rs{avg_profit_per_trip}
00112 - Top Routes: {top_routes}
00113 """
00114
00115 """
00116
     ------------------------------------------------------------------------------------------------------------------------
00117 Main Application Class
00118
     ------------------------------------------------------------------------------------------------------------------------
00119 [Summary] TripAuditorApp is the main Flask application for managing trip audits.
00120 [Details] It initializes the Flask app, loads datasets, manages user sessions, and defines all routes
     for the application.
00121
00122 """
00123
00124 class TripAuditorApp:
00125     # This method initializes the Flask app, loads datasets, prepares vehicle and route lists, and
     sets up user management.
00126     def __init__(self):
00127         # Initialize Flask app
00128         self.app = Flask(__name__)
00129         self.app.secret_key = 'supersecret'
00130
00131         # Load datasets
00132         self.fleet_file = 'fleet_50_entries.xlsx'
00133         self.closure_file = 'Trip_Closure_Sheet_Oct2024_Mar2025.xlsx'
00134         self.df = self.load_fleet_data()
00135         self.closure_df = self.load_closure_data()
00136
00137         # Prepare vehicle and route lists
00138         self.vehicles = sorted(self.df['Vehicle ID'].dropna().unique())
00139         self.routes = sorted(self.df['Route'].dropna().unique()) if 'Route' in self.df.columns else []
00140
00141         # User management
00142         self.user_manager = UserManager(os.path.join(os.getcwd(), 'users.json'))
00143
00144         # Register all routes
00145         self.register_routes()
00146
00147     # This method loads and preprocesses the fleet Excel data, converting date columns and cleaning up
     column names.
00148     def load_fleet_data(self):
00149         """Load and preprocess the fleet Excel data."""
00150         df = pd.read_excel(self.fleet_file)
00151         df.columns = df.columns.str.strip()
00152         df['Trip Date'] = pd.to_datetime(df['Trip Date'], errors='coerce')
00153         df['Day'] = df['Trip Date'].dt.day
00154         return df
00155
00156     # This method loads and preprocesses the closure Excel data, converting date columns and cleaning
     up column names.
00157     def load_closure_data(self):
00158         """Load and preprocess the closure Excel data."""
00159         df = pd.read_excel(self.closure_file)
00160         df.columns = df.columns.str.strip()
00161         df['Trip Date'] = pd.to_datetime(df['Trip Date'], errors='coerce')
00162         df['Day'] = df['Trip Date'].dt.day
00163         return df
00164
00165     # This method registers all Flask routes for the application, defining the logic for each route.
00166     def register_routes(self):
00167         """Define all Flask routes for the app."""
00168
00169         @self.app.route('/')
00170         def home():
00171             # Redirect to signup page
00172             return redirect(url_for('signup'))
00173
00174         @self.app.route('/signup', methods=['GET', 'POST'])
00175         def signup():
00176             # Handle user signup
```

```
00177             if request.method == 'POST':
00178                 email = request.form['email']
00179                 if self.user_manager.email_exists(email):
00180                     return render_template('signup.html', error="Email already registered!")
00181                 self.user_manager.add_user(
00182                     name=request.form['fullname'],
00183                     email=email,
00184                     password=request.form['password']
00185                 )
00186                 return redirect(url_for('login'))
00187             return render_template('signup.html')
00188 
00189         @self.app.route('/login', methods=['GET', 'POST'])
00190         def login():
00191             # Handle user login
00192             if request.method == 'POST':
00193                 user = self.user_manager.find_user(request.form['email'])
00194                 if user and check_password_hash(user['password'], request.form['password']):
00195                     session['user'] = user
00196                     return redirect(url_for('dashboard'))
00197                 return render_template('login.html', error="Invalid credentials.")
00198             return render_template('login.html')
00199 
00200         @self.app.route('/dashboard')
00201         def dashboard():
00202             # Main dashboard with filters and summary
00203             if 'user' not in session:
00204                 return redirect(url_for('login'))
00205             vehicle = request.args.get('vehicle')
00206             route = request.args.get('route')
00207             filtered = self.df.copy()
00208             if vehicle:
00209                 filtered = filtered[filtered['Vehicle ID'] == vehicle]
00210             if route:
00211                 filtered = filtered[filtered['Route'] == route]
00212 
00213             total_trips = len(filtered)
00214             ongoing = filtered[filtered['Trip Status'] == 'Pending Closure'].shape[0]
00215             closed = filtered[filtered['Trip Status'] == 'Completed'].shape[0]
00216             flags = filtered[filtered['Trip Status'] == 'Under Audit'].shape[0]
00217             resolved = filtered[(filtered['Trip Status'] == 'Under Audit') & (filtered['POD Status']
    == 'Yes')].shape[0]
00218 
00219             rev = filtered['Freight Amount'].sum()
00220             exp = filtered['Total Trip Expense'].sum()
00221             profit = filtered['Net Profit'].sum()
00222             kms = filtered['Actual Distance (KM)'].sum()
00223 
00224             rev_m = round(rev / 1e6, 2)
00225             exp_m = round(exp / 1e6, 2)
00226             profit_m = round(profit / 1e6, 2)
00227             kms_k = round(kms / 1e3, 1)
00228             per_km = round(profit / kms, 2) if kms else 0
00229             profit_pct = round((profit / rev) * 100, 1) if rev else 0
00230 
00231             daily = filtered.groupby('Day')['Trip ID'].count().reindex(range(1, 32),
    fill_value=0).tolist()
00232             audited = filtered[filtered['Trip Status'] == 'Under Audit'].groupby('Day')['Trip
    ID'].count().reindex(range(1, 32), fill_value=0).tolist()
00233             audit_pct = [round(a / b * 100, 1) if b else 0 for a, b in zip(audited, daily)]
00234 
00235             bar_labels = ['Revenue', 'Expense', 'Profit']
00236             bar_values = [float(rev_m), float(exp_m), float(profit_m)]
00237             ai_report = AIReportGenerator.generate(filtered)
00238 
00239             return render_template('dashboard.html',
00240                 total_trips=total_trips, ongoing=ongoing, closed=closed,
00241                 flags=flags, resolved=resolved, rev_m=rev_m, exp_m=exp_m,
00242                 profit_m=profit_m, kms_k=kms_k, per_km=per_km, profit_pct=profit_pct,
00243                 ai_report=ai_report, vehicles=self.vehicles, routes=self.routes,
00244                 selected_vehicle=vehicle, selected_route=route,
00245                 daily=daily, audited=audited, audit_pct=audit_pct,
00246                 bar_labels=bar_labels, bar_values=bar_values)
00247 
00248         @self.app.route('/trip-generator')
00249         def trip_generator():
00250             # Show all trips
00251             data = self.df[['Trip ID', 'Vehicle ID', 'Trip Status']]
00252             return render_template('table_page.html', title="Trip Generator",
    table=data.to_html(classes='text-white', index=False))
00253 
00254         @self.app.route('/trip-closure')
00255         def trip_closure():
00256             # Show trips pending closure
00257             data = self.df[self.df['Trip Status'] == 'Pending Closure'][['Trip ID', 'Vehicle ID',
    'Trip Status']]
00258             return render_template('table_page.html', title="Trip Closure",
```

```
    table=data.to_html(classes='text-white', index=False))
00259
00260         @self.app.route('/trip-auditor')
00261         def trip_auditor():
00262             # Show trips under audit
00263             data = self.df[self.df['Trip Status'] == 'Under Audit'][['Trip ID', 'Vehicle ID', 'POD
    Status']]
00264             return render_template('table_page.html', title="Trip Auditor",
    table=data.to_html(classes='text-white', index=False))
00265
00266         @self.app.route('/trip-ongoing')
00267         def trip_ongoing():
00268             # Show ongoing trips
00269             data = self.df[self.df['Trip Status'] == 'Pending Closure'][['Trip ID', 'Vehicle ID',
    'Trip Status']]
00270             return render_template('table_page.html', title="Ongoing Trips",
    table=data.to_html(classes='text-white', index=False))
00271
00272         @self.app.route('/trip-stats')
00273         def trip_stats():
00274             # Show trip statistics by day
00275             days = list(range(1, 32))
00276             total = self.df.groupby('Day')['Trip ID'].count().reindex(days, fill_value=0).tolist()
00277             ongoing = self.df[self.df['Trip Status'] == 'Pending Closure'].groupby('Day')['Trip
    ID'].count().reindex(days, fill_value=0).tolist()
00278             closed = self.df[self.df['Trip Status'] == 'Completed'].groupby('Day')['Trip
    ID'].count().reindex(days, fill_value=0).tolist()
00279
00280             total_json = json.dumps(total)
00281             ongoing_json = json.dumps(ongoing)
00282             closed_json = json.dumps(closed)
00283
00284             total_sum = sum(total)
00285             ongoing_sum = sum(ongoing)
00286             closed_sum = sum(closed)
00287
00288             return render_template('trip_stats.html',
00289                 total_data=total_json, ongoing_data=ongoing_json, closed_data=closed_json,
00290                 total_sum=total_sum, ongoing_sum=ongoing_sum, closed_sum=closed_sum)
00291
00292         @self.app.route('/financial-dashboard')
00293         def financial_dashboard():
00294             # Show financial dashboard with recent 10 days data
00295             df_fin = self.closure_df.copy()
00296             recent_days = sorted(df_fin['Day'].dropna().unique())[-10:]
00297             day_labels = [f"Day {int(d)}" for d in recent_days]
00298             daily = df_fin[df_fin['Day'].isin(recent_days)]
00299             revenue_data = daily.groupby('Day')['Freight Amount'].sum().reindex(recent_days,
    fill_value=0).astype(int).tolist()
00300             expense_data = daily.groupby('Day')['Total Trip Expense'].sum().reindex(recent_days,
    fill_value=0).astype(int).tolist()
00301             profit_data = [r - e for r, e in zip(revenue_data, expense_data)]
00302             total_revenue = round(df_fin['Freight Amount'].sum() / 1e6, 2)
00303             total_profit = round(df_fin['Net Profit'].sum() / 1e6, 2)
00304             total_km = round(df_fin['Actual Distance (KM)'].sum() / 1e3, 1)
00305             per_km = round(df_fin['Net Profit'].sum() / df_fin['Actual Distance (KM)'].sum(), 2) if
    df_fin['Actual Distance (KM)'].sum() else 0
00306
00307             return render_template('financial_dashboard.html',
00308                 days=day_labels, revenue=revenue_data, expense=expense_data, profit=profit_data,
00309                 total_revenue=total_revenue, total_profit=total_profit, total_km=total_km,
    per_km=per_km)
00310
00311         @self.app.route('/logout')
00312         def logout():
00313             # Log out the current user
00314             session.pop('user', None)
00315             return redirect(url_for('login'))
00316
00317         @self.app.route('/download-summary')
00318         def download_summary():
00319             # Download AI report summary as a text file
00320             filtered = self.df
00321             report = AIReportGenerator.generate(filtered)
00322             with open("AI_Report_Summary.txt", 'w', encoding='utf-8') as f:
00323                 f.write(report)
00324             return send_file("AI_Report_Summary.txt", as_attachment=True)
00325
00326     def run(self):
00327         """Run the Flask app."""
00328         self.app.run(host='0.0.0.0', port=7860, debug=True)
00329
00330 """
00331
    ---------------------------------------------------------------------------------------------------------------------------
00332 Entry Point
00333
```

```
    ------------------------------------------------------------------------------------------------------------------------
00334 This is the entry point for the application, where the TripAuditorApp is created and run.
00335
00336 """
00337
00338 if __name__ == "__main__":
00339     # Create and run the OOP-based TripAuditor app
00340     app = TripAuditorApp()
00341     app.run()
```